# Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package

Kamen Levi
Independent researcher

9 November 2026

**Abstract**

This paper documents the engine package behind [MAIN]. The engine runs Qwen3.8-Flash-Next, a 125B mixture-of-experts model, on one MacBook Pro (M5 Pro, 18-core CPU, 20-core GPU, 64 GB, 2 TB SSD): the experts are stored once on the internal SSD, and each layer keeps a fixed pool of them in memory. It installs from a clean clone (tested on one machine) and ships six configurations: exact, lossy and chat, each at a 45 GB and a 32 GB memory line, labelled as [MAIN] section 2 defines. Package 1.1.0 adds a 24 GB line: one gated lossy configuration, under the names lossy and chat (section 2; [SMALL-RAM]). It also adds a chat in the terminal and a local server for chat apps in two common API formats, text only (section 4). At 45 GB, the exact configuration, bit-identical to one-token greedy decoding under our reference NAX6-N, decodes 37.1-101.4 tok/s; the shipped lossy one, which scored the same as the exact reference on the model card's IFBench, GPQA Diamond and LiveCodeBench v6 in both orders, decodes 45.2-164.0 tok/s; both stay at a lifetime peak of 41.7-43.2 GB, plus a 7.45 GB expert tier that macOS may reclaim [K, section 2]. Where we lose: on the same machine and prompts, Strata's first token (0.5-1.0 s) beats ours (2.7-4.6 s; 2.26 s on chat as shipped), so up to 256 generated tokens Strata wins or ties every whole-request number [K, section 2]. The code is GPL-3; the model's weights keep their own licence (section 7).

Every number has a mark ([K] measured, [P] priced, [G] judgement) and a source key (section 8; keys as in [MAIN] Appendix C). Code: GPL-3.

## 1. What this is

An inference engine that runs Qwen3.8-Flash-Next, a 125B mixture-of-experts model (48 layers, 512 routed experts per layer, 10 per token), on one MacBook Pro (M5 Pro, 64 GB unified memory, internal SSD). The experts are stored once on the SSD at about 3.2 bits per expert weight; each layer keeps a fixed pool of experts in memory and reads the rest on demand. The engine's lifetime peak memory footprint is measured against the memory line it is named for (45, 32 or 24 GB): the engine's own footprint on a 64 GB machine, not an OS limit ("GB" = `10^9` bytes of the process's lifetime peak phys_footprint). Besides it, every configuration at 45 and 32 GB holds a 7.45 GB volatile expert tier as purgeable memory, which macOS reclaims before it compresses or swaps and which phys_footprint does not count [K, BM:data/rel/rli_160.txt line 7]; the 24 GB one holds none [K, BM:data/stack/card_w24d_env.txt].

It ships three kinds of configuration, labelled as [MAIN] section 2 defines them:

- **exact (to NAX6-N):** output tokens and the logits behind every kept token are bit-identical to one-token greedy decoding under our reference NAX6-N.
- **lossy, same scores on :** passed a pre-registered same-scores gate against the exact reference on the named known benchmarks ([EVAL]).
- **lossy, ungated:** faster, not yet gated; shipped switched off, labelled.

## 2. Results you should expect (pooled; 12 prompts per set, 256 new tokens, greedy; [MAIN] has the full tables)

Whole decode / whole request (tok/s) / first token (median, s) / lifetime peak (GB). Source rows: section 8.

| line | config | label | HumanEval | MBPP | IFEval (chat) | math |
|---|---|---|---|---|---|---|
| 45 GB | E (+ bundle) | exact (NAX6-N) | 101.4 / 32.6 / 4.41, 42.24 [K] | 70.4 / 21.7 / 3.80, 41.92 [K] | 37.1 / 20.8 / 2.89, 41.70 [K] | 77.9 / 38.9 / 2.73, 42.50 [K] |
| 45 GB | lossy as shipped: L + COH | lossy, same scores on IFBench, GPQA Diamond and LiveCodeBench v6 (the card gate, both orders, vs NAX6-N; EV:RESULT_card_lcoh_2026-11-09; the package runs the gated code and env, section 4). The certificate (next row's label) does not carry: it was earned without the bundle and without COH, and COH changes the one-call path's reads. L alone is run-to-run nondeterministic on chat; L + COH repeated itself: a second IFEval pass in its own process, same prompts in the same order, gave the same tokens on 12 / 12 prompts [K]. One pass on HumanEval, MBPP and math; IFEval pools the two passes | 164.0 / 36.2 / 4.60, 42.94 [K] | 112.7 / 25.3 / 3.72, 42.60 [K] | 45.2 / 22.4 / 3.01, 42.42 [K] | 78.9 / 37.9 / 3.20, 43.17 [K] |
| 45 GB | L = the certified flags (CERT: v1' + wired) + the exact-safe bundle; row 27 without COH; not shipped | lossy, not gated by itself. The certificate (same scores on HumanEval+, MBPP, math and IFEval, both orders; EV:RESULT_cert_2026-11-08) was earned by CERT, without the bundle. CERT's own rows (S5' B): HumanEval 171.0 / 36.0 / 4.56, 42.91 [K]; MBPP 114.5 / 25.8 / 3.79, 42.55 [K]. The bundle is bitwise on the exact path; on this lossy path it is not shown to keep tokens | 181.3 / 36.6 / 4.51, 42.91 [K] | 119.8 / 25.3 / 3.76, 42.57 [K] | 43.4 / 21.1 / 3.00, 42.37 [K] | 86.9 / 39.6 / 2.96, 43.10 [K] |
| 45 GB | M1-B + COH (chat) | lossy, same scores on IFEval (both orders; EV:RESULT_chat_m1b_2026-11-08) and on IFBench, GPQA Diamond and LiveCodeBench v6 (the card gate, both orders, vs NAX6-N; EV:RESULT_card_m1b_2026-11-09); the multilingual set is information only | - | - | 49.8 / 24.4 / 2.92, 42.38 [K] | - |
| 45 GB | chat as shipped: zs (M1-B + COH + SB1) + PF-D v4 + read-ahead | as M1-B + COH: the same tokens on all 12 prompts [K]; PF-D v4 is bitwise to zs (logits hashes 721/721) and SB1 only reorders work (logits hashes 676/676 on E) [K] | - | - | 50.7 / 28.8 / 2.26, 43.69 [K] (one pass) | - |

| line | config | label | HumanEval | MBPP | IFEval (chat) | math |
|---|---|---|---|---|---|---|
| 32 GB | E at C 160 | exact (NAX6-N) | not measured | not measured | 33.8 / 19.8 / 2.83, 30.62 [K] | not measured |
| 32 GB | L + COH at C 160 (lossy-32) | lossy, not gated (no gate ran at C 160); C30's PASS (L at C 160 before COH; HumanEval+ only, EV:RESULT_c30_ 2026-11-07) does not carry: COH changes tokens on the one-call path; the card gate (row 27) ran at C 256 only. On IFEval, C 160 gives the same tokens as C 256 (12 / 12) [K] | C30, before COH: 150.7 / 36.0 / 4.43, 31.78 [K] | C30, before COH: 111.0 / 26.8 / 3.69, 31.44 [K] | 39.0 / 21.1 / 2.90, 31.30 [K] (one pass) | - |
| 32 GB | zs32 = M1-B + COH + SB1 at C 160 | lossy, not gated: its card gate did not fit in Monday's gate window (EV:PREREG_card_ gates_2026-11-08, amendment 11); row 29's IFEval pass was at C 256 | - | - | 51.3 / 22.5 / 2.99, 31.22 [K] | - |
| 24 GB | w13b = L + COH with 141 + 26 expert slots per layer and checks of at most 13 rows, at C 100; shipped as both lossy and chat (1.1.0) | lossy, same scores on IFBench, GPQA Diamond and LiveCodeBench v6 (the 24 GB card gate, both orders, vs NAX6-N; EV:RESULT_card_ w24_2026-11-10). Two passes pooled per set. Under 24 GB in our runs on prompts of up to about 500 tokens, with little room (below) | 130.3 / 40.8 / 3.54, 23.82 [K] | 91.2 / 26.1 / 3.34, 23.74 [K] | 30.6 / 18.1 / 3.09, 23.66 [K] | 40.8 / 27.4 / 2.86, 24.01 [K] |

- The shipped lossy config's card gate (L + COH against the exact reference, both orders): IFBench 40.7 against 44.0 % (-3.3 points), GPQA Diamond 47.0 against 44.0 % (+3.0), LiveCodeBench v6 30.5 / 29.0 against 32.1 % (-1.5 / -3.1). All are inside the pre-registered rule (a delta above -5 points and an exact McNemar p of at least 0.05), so it passes. A pass bounds the loss at these sizes (150 / 100 / 131 questions); it does not show there is none [K, EV:RESULT_card_lcoh_2026-11-09].
- The chat config's card gate (M1-B + COH against the exact reference, forward / reverse order): IFBench 44.0 / 41.3 against 44.0 % (0.0 / -2.7 points), GPQA Diamond 42.0 / 42.0 against 44.0 % (-2.0 / -2.0), LiveCodeBench v6 29.0 / 32.1 against 32.1 % (-3.1 / 0.0). All six are inside the same rule, so it passes; the same bound applies [K, EV:RESULT_card_m1b_2026-11-09].
- The shipped lossy config decodes slower than L (row 28) on code and math: HumanEval 164.0 against 181.3 tok/s, MBPP 112.7 against 119.8, math 78.9 against 86.9; on IFEval it is faster, 45.2 against 43.4 [K]. This is not an A/B: the runs are from different days (8 and 9 Nov), L + COH has one pass on those three sets against L's two, and COH changes tokens, so acceptance differs too. The cause is not measured.
- Where we lose: against Strata (MIT; github.com/Niko1221/Strata), run by us on the same machine and prompts (every Strata number here is our measurement, none self-reported), our decode is ahead on code and math and about level on chat, exact chat decode is behind, and Strata's first token (0.5-1.0 s) beats ours (2.7-4.6 s; 2.26 s on chat as shipped, with PF-D), so up to 256 generated tokens Strata wins or ties every whole-request number. Chat as shipped (2.26 s, 28.8 tok/s whole request, peak 43.7 GB) against Strata's full-cache arms (0.83 s, 33.0-33.7 tok/s; peaks 40.8-41.2 GB): first token 2.7x slower, Strata ahead on whole request, our chat decode ahead (50.7 vs 40.6-41.8 tok/s) [K, SE:h2h/results_ arena33/{B1,D1}.requests.jsonl].
- Long prompts: the default prefill re-reads each layer's experts per 1,024-token chunk. PF-A (layer-major, on prompts over 1,024 tokens; exact except when the last 1,024-token chunk is under about 200 tokens [K, LP:data/pfa_tail_1111.md]) and PF-B (layer split and read-ahead; exact on the 1.6K-7.9K-token

agent prompts since the seg-per-chunk fix, not on prompts whose last chunk is short, such as a 30K-token prompt; its fix is not in this package ([MAIN] section 6.2)) prefill a 7.9K-token agent prompt at 433 and 500 tok/s [K] (exact E; lifetime peak 45.09 / 46.23 GB [K], above the 45.0 line on that prompt). At 32 and 24 GB, chat.py runs PF-A by default (section 4).

- Chat first token: PF-D builds a chat prompt's pool-resident experts on the GPU from memory instead of re-reading them from the SSD; v4 also reads the next layer's misses during this layer, as Strata's prompt path does (section 7). It is exact: tokens and logits are bitwise equal, and every staged row equals the reference rebuild (87,912 rows on E) [K]. On the chat config the first token falls from 2.91 to 2.26 s (median), whole request rises from 24.6 to 28.8 tok/s and whole decode is unchanged (50.3 / 50.7) [K, 12 IFEval, 256 tokens, one pass each]. On exact E an earlier version cut the first token from 2.89 to 2.26 s (64 tokens) [K]. Default: on for chat at 45 GB. It is off for the other configs, whose rows above were measured without it, and it was not measured at 32 GB, where its 1.0-1.3 GB of extra peak matters. An earlier run's decode gap (40.0 against 49.6 tok/s) did not reproduce when the PF-D arm ran first; its cause is unknown [K]. A lossy switch (absent experts' side parts skipped: 3-bit weights) reaches 1.83 s and 30.5 tok/s but changes tokens on 10 of 12 prompts; it is ungated and ships off.
- The 24 GB line (package 1.1.0; [SMALL-RAM] has its full tables, its gate and its memory analysis): lossy-24 and chat-24 are one env line, the gated one, with the gated runs' decode loop [K, LP:data/w24_pkg.md]. In our runs it stayed under 24 GB on prompts of up to about 500 tokens, with little room, so this is a measurement, not a bound: a 448-token gate prompt raised its process's peak to 23.954 GB [K, EvalSuite/results/card/es_cdw24_100136_rev17_runs.json], a 520-token math prompt reached 24.01 GB [K, BM:data/r57/bg1a_100_runs.json], and one 592-token prompt, first in a fresh process both times, peaked 0.16 GB apart (23.954 and 24.112 GB) [K, EvalSuite/results/card/es_cdw24_100136_fwd18_runs.json; EvalSuite/results/card/es_cdw24_100136_rev3_runs.json]. Longer prompts use more: 25.0 GB at 3K tokens (on the simulated 36 GB Mac, section 3), 25.7 at 7K, 25.8 at 10K and 28.2 at 30K (default prefill, as run.py runs it) [K, LP:data/tue_afternoon_1110.md sections 1, 2 and 4]. The package's lossy-24 gave the research runs' tokens on all 12 HumanEval prompts and the gate's on six of its IFBench and GPQA Diamond prompts, at 135.9 tok/s whole decode on HumanEval (one pass, under a share lock) and a 23.78 GB peak [K, LP:data/w24_pkg/check_summary.txt]. Where we lose: Strata at about the same peak (23.75-24.14 GB), on the same Mac and prompts, has the faster first token on every set (1.31-2.31 s against our 2.86-3.54 s) and the faster whole request on chat (20.3-21.0 against 18.1 tok/s); our whole decode and whole request lead on code and math [K, SE:h2h/results_low24].

# 3. Requirements

- Apple silicon Mac with 64 GB unified memory (tested: MacBook Pro, M5 Pro, 64 GB); tested on macOS 26.5 (25F71) only.
- The 24 GB line is meant for a Mac with 36 GB of memory, but it was not run on one. It ran only on the Mac above, once with a 36 GB Mac simulated on it: memory locked away left the engine 25-27 GB, and its GPU working set was capped at 27 GB. There it gave the same tokens on the 12 HumanEval prompts at 137.3-138.1 tok/s whole decode, and a 3,003-token prompt peaked at 25.0 GB, with swap flat or up 0.4 GB [K, LP:data/tue_afternoon_1110.md section 1]. A real 36 GB Mac has its own GPU, SSD, thermals and memory policy, and MLX picks some kernel tiles by GPU type, so its tokens and speed may differ [G].
- Python 3.12; MLX pinned to 0.32.2 with the pad-to-64 fix for MLX issue #4632 (sorted gather_qmm leaves rows unwritten past 32,767 when M is not a multiple of 64; 0.32.4 fixes it but changes tokens against our exact references); mlx-lm 0.31.3; MTPLX 2.12.2 from a pinned commit (section 4); CMake + a C++ toolchain for the extension.
- The model weights: Qwen3.8-Flash-Next by Qwen (Qwen Community License 1.0, section 7), as MTPLX's flat 4-bit MLX pack (Hugging Face: Youssofal/Qwen3.8-Flash-Next-MTPLX-Bare-Speed). The package ships no weights.
- Disk: the pack as downloaded, without its vision tower: 105.4 GB [K] (routed experts 67.9 GB; the n-gram table, 32.0 GB, which the model code streams from the SSD at run time; dense text weights 3.8 GB; MTP head 1.7 GB), plus the converted expert store, 48 x 512 experts x 2.79 MB = 68.5 GB [K] (records3, native+p11+pad layout): about 174 GB in all [K].

# 4. Install and run

**The package** is the frozen import closure of real runs, not a rewrite: the engine's frozen snapshot (the decode loop and every module it loaded, taken read-only from the commit that holds the snapshot), the union of all 174 module manifests the engine's runs wrote (43 modules from the kernel, drafter and keep-set trees), the engine package, the extension source, the harness package and the published env lines. The manifests miss three kernel files that the lossy and chat runs read from another research folder (section 8); the package already held byte-identical copies, and the clean-clone smoke found that the engine still looked for them in that folder (below). A build script renames the trees (engine/, kernels/, third_party/, drafters/, keepset/, harness/), rewrites every path into the clone, scrubs comments by the public snapshot's rules and refuses to commit on a leak, a syntax error or a path left pointing at the research trees. Its research-path check did not cover that folder until the smoke: the two engine lines that name it are now rewritten to the package's copies, and the build lists the 43 other home-folder paths still named in the code (records of a run's source, benchmark outputs, optional self-checks, switches the harness refuses) in its manifest. The build from the PF-D snapshot: 218 files, 6.2 MB, 0 leaks, 0 syntax errors, 0 research paths left by the check of that day; from the provisional release freeze rel0 after the smoke's fixes (9 Nov 07:20): 222 files, 6.3 MB, 0 / 0 / 0 [K, LT:'ENGINE PACKAGE v0']; the release build from the engine's final snapshot rel1 (frozen 9 Nov 07:19, identical to rel0 apart from its name), 9 Nov 09:57: 225 files, 6.3 MB, 0 / 0 / 0, which are rel0's files plus the conversion driver and the two lossy references [K, LP:data/release/rel1_build.txt].

**Install** (Python 3.12, CMake, the Xcode command-line tools):

```
bash install.sh --store <model dir with records3/> --weights <dir with l120_ft.safetensors and
d22_b5.safetensors>
```

It writes the clone's path into the code, creates a venv with the pinned packages (requirements.txt: MLX 0.32.2, mlx-lm 0.31.3, MTPLX 2.12.2 at its GitHub commit, and the rest of the measured venv), builds the extension with the vendored nanobind 2.15.0, and links the store and the two drafter files: l120_ft.safetensors (1.14 GB, our finetune of DFlash2) and d22_b5.safetensors (277 MB, the MTP head). Neither is in the repository; how they are published is decided with the release. The block drafter's base checkpoint (DFlash2, section 7) comes from the Hugging Face cache at the revision our runs loaded: install.sh checks for it, and run.py pins it (the drafter's own loader would take the cache's default branch) and stops with a one-line fetch command when it is missing. A cache that holds only that revision resolves through run.py and fails without the pin [K, LP:data/release/drafter_pin_test.txt].

**Run:**

```
.venv/bin/python run.py --config exact|lossy|chat --mem 45|32|24 --prompt "..." [--max-tokens 256]
```

run.py runs the decode loop unchanged, the way the research runner ran it (one arm, one repetition, one discarded warm-up prompt), with the config's env line from configs/-.env. It only turns a text prompt into a prompt item with the harness's chat template (non-thinking) and prints the answer, whole decode, whole request, first token and the lifetime peak. `-keys ifeval/1220,...` runs benchmark items instead. The six configs: exact = E (exact to NAX6-N) at C 256 / 160; lossy = L + COH; chat = zs (M1-B + COH + SB1, bitwise equal to M1-B + COH), at 45 GB with PF-D v4 and its read-ahead (R57_PFD=1, R57_PFD_V=4, R57_PFD_AHEAD=1; the same tokens, section 8), at 32 GB without; PF-D is off in the other four; their labels are section 1's. Each env line is the published one (parse-checked against the engine's run scripts) [K, LT:'ENGINE PACKAGE v0']. At 24 GB (1.1.0) there are only lossy and chat, and both are w13b: the 24 GB gate's env line (its 42 keys parse equal) at C 100, plus a line that picks the decode loop the gate ran [K, LP:data/w24_pkg.md]. The 45 and 32 GB lossy configs run what the card gate ran: against the card snapshot the engine package, the extension source and every module the lossy runs import are the same, the decode loop differs only by the PF-D switch (off in them), and the env line differs only by the name of the card's own code benchmark in the QOM sets (run.py names a code prompt `humaneval`, which is in them, and a chat prompt `multi`, which is not) [K, LP:data/release/lossy_card_equiv.txt].

**Conversion.** The store is built in three steps: (a) safetensors -> records/ (4-bit pieces; repack_records.py), (b) records/ -> records3 row-major, MSB-3 + LSB-1 parts (make_records3.py, with a verifier on every expert: rebuilt 4-bit == the originals, s3 / 2 == s), (c) row-major -> native+p11+pad (convert_records_p11.py --pad); format: DESIGN_3bit.md "Record file". convert/convert_store.sh runs the three steps layer by layer into a store (a layer's 4-bit pieces are deleted after the split; finished layers are skipped when it is run again). Run from the tested package clone on all 48 layers of the safetensors (9 Nov 16:07-16:17), it built the full store in 9.5 min (11-14 s per layer), and the result equals the store that every run in this paper used, byte for byte: all 48 layer files (1,426,063,360 B each) and index.json [K, LP:data/release/convert_full.txt]. Earlier runs on layers 0 and 47 alone gave the same bytes, at 19-39 s per layer with other jobs running [K, LP:data/release/

pfd_convert_report.txt, LP:data/release/convert_store_test.txt]. The release runs used the existing store.

**Clean-clone test (passed on one machine).** `tools/clean_clone_test.sh` clones the package, installs it offline (no network: the wheels are rebuilt from the measured venv's installed files; MTPLX's 321 source files are identical to the venv's), builds the extension, then runs every config on two IFEval prompts (up to 256 tokens each, the references' cap) with HOME set to an empty directory, so any path left pointing outside the clone fails instead of reading the research trees. It checks the tokens against the shipped references (the exact configs as a prefix of one-token decoding; chat and lossy against the engine's own runs of the same config: for lossy, its IFEval runs of L + COH on rel1 at C 256 and C 160, which a repeat in its own process matched on 12 / 12 prompts and which equal each other on 12 / 12 [K, BM:data/rel/rel_rows_report.txt]; a reference of a config never repeated would ship as information only), that every module the engine imported lies in the clone or its venv, and that every file Python opened lies in the clone, its venv, the store, the drafter files, the Hugging Face cache or the system's folders. The lossy and chat configs' tokens depend on what the process decoded before (their relaxed acceptance takes the drafter's proposals, and the drafter's state carries across prompts), so a prompt of theirs is checked only when the earlier prompts ran as in the reference run: the same prompts, in the same order, to the same length. Without the GPU (8 Nov): install rc 0 in 5-7 s, dry runs of all six configs, and all 51 shipped modules import from the clone under the empty HOME with none from outside it [K, LP:data/release/pfd_prep_report.txt]; the rel0 build again: rc 0 in 7 s, six dry runs [K, LP:data/release/rel0_prep_report.txt]. The GPU smoke on rel0, 9 Nov 07:00-07:32 [K, LP:data/release/smoke_0700.txt]:

- 07:01: the exact configs' tokens equal their references on both prompts, but the four lossy and chat configs and the free-text prompt stopped at import: the engine still looked for the three kernel files in the other research folder (section 8), and the closure check flagged the drafter files because it followed their links without allowing them (a fault of the check). Both fixed in the build: the two path strings now point at the shipped copies, and the check allows the linked drafter files.
- 07:13, at a 64-token cap: all six configs and the free-text prompt ran, with every module and file inside the allowed folders; exact 2 / 2; chat matched prompt 1 but not prompt 2 (from token 4 at 45 GB, token 60 at 32 GB), because the cap cut prompt 1 at 64 tokens where the reference run let it end at 192 / 184, so prompt 2 followed a different history.
- 07:28, at cap 256 with the history rule above: CLEAN-CLONE PASS. Exact 2 / 2; chat 2 / 2 to the references' full length (192 + 235 and 184 + 256 tokens); lossy reported (no reference yet); every module and every opened file inside the allowed folders on all six configs; the free-text prompt gave a complete answer.

Before the final run, the 07:28 smoke's lossy tokens (rel0, the same code) were compared on the CPU with the L + COH references: 2 / 2 prompts equal to their full length (154 + 139 tokens) at 45 and 32 GB [K, LP:data/release/lossy_precheck.txt]. The final run on the release package (rel1, its clone prepared 9 Nov 09:58), with those references as checks, ran 9 Nov 12:10-12:14: CLEAN-CLONE PASS [K, LP:data/release/final_1210.txt]. All six configs gave their references' tokens on both prompts (exact as a prefix of one-token decoding; lossy and chat to the references' full length), every module and every opened file on all six lay inside the allowed folders (834-849 modules and 3,268-3,353 files per config, none outside), and the free-text prompt gave the same complete 87-token answer as at 07:28. Read on the CPU afterwards, its tokens equal the 07:28 smoke's on all 12 prompt runs. On the lossy and chat configs each prompt took the same checks and kept the same tokens per check as the engine's own run of it, at the same pace: over the two prompts, whole decode x0.996-1.020 and whole request x0.996-1.012 of those runs, median first token within 0.11 s (information only: two prompts per config) [K, LP:data/release/final_1210.txt].

Package 1.1.0's engine passed the same test on 10 Nov (10:18-10:23): all eight configs gave their references' tokens on two prompts each, the 24 GB pair on HumanEval against the research runs, with every module and opened file inside the allowed folders [K, LP:data/w24_pkg/candidate/report.txt]. The 1.1.0 package differs from that build in its documents, the memory comment in its two 24 GB configs, one help string and two new programs, chat.py and serve.py, which the test script now also runs (below) [K, LP:data/w24_pkg.md].

**Chat and a local server (1.1.0).** Package 1.1.0 adds two programs that load the engine once and run every answer through run.py's decode loop, unchanged; both default to the chat configuration at 32 GB (code: LP:release/src/chat.py and LP:release/src/serve.py). `chat.py` is a chat in the terminal: answers print as they are written, with the live speed in the window title; a paste is one message at any length, and `/file PATH message` sends a file, then the message. A built-in system line tells the model that it has no tools. `serve.py` serves chat apps on the same Mac (127.0.0.1) in two API formats, OpenAI's chat completions format (`/v1/chat/completions`) and the messages format (`/v1/messages`), with streaming, one request at a time. In 1.1.0 it handles text only: tool definitions and tool calls are not mapped, so it cannot serve an agent that calls

tools.

- At 32 and 24 GB chat.py prefills prompts over 1,024 tokens layer-major (PF-A, section 2; `R57_PFLM=0` in the shell turns it off); run.py and serve.py do not. On the chat configuration PF-A brings the first token 3.5-3.7x sooner at 32 GB (10K- and 30K-token prompts) and 2.4-2.7x sooner at 24 GB (7K-30K tokens), for 0.3-3.0 GB more lifetime peak (section 5) [K, LP:data/prefill_1109.md; LP:data/tue_afternoon_1110.md sections 2 and 4]. Its tokens are the default prefill's except when the last 1,024-token chunk is under about 200 tokens [K, LP:data/pfa_tail_1111.md].
- Both make the extension's loader thread nap after 50 ms without work instead of spinning, so an idle chat or server no longer keeps a core busy. Idle after a request (CPU time over 60 s), serve.py used a full core without the nap and 1.5-1.6 % of one with it, at 24 and 32 GB [K, LP:data/agent_cc_1111.md; LP:data/idle_ms_1111.md]; the candidate build 110g, whose code ships unchanged in package 1.1.0, idled with its own defaults at 1.6 % (serve.py) and 1.5 % (chat.py) at 24 GB [K, LP:data/w24_pkg.md]. Whole decode did not measurably change: x1.002 at 24 GB (two runs per arm) and x0.995 at 32 GB (five and six runs; one more ran through a heat pause and is left out), with the same tokens in every run (three HumanEval prompts) [K, LP:data/idle_ms_1111.md].
- Checks: in build 110g's clean-clone test (11 Nov 03:52-03:54), lossy-24 and chat-24 gave their references' tokens with every module and opened file inside the allowed folders, chat.py answered a message at 24 GB (lifetime peak 23.63 GB), and serve.py at 24 GB answered a streamed request in each format and stopped on SIGTERM [K, LP:data/w24_pkg.md; LP:data/w24_pkg/candidate_110g/checks_0345/clean_clone_run.log]. chat.py's interactive parts (pastes, `/file`, the window title) passed a terminal test against a stand-in for the engine, 22 checks of 22, and chat.py ran twice on the real engine at 32 GB [K, LP:data/chatpy_fixes_1110.md].

# 5. Known limits

- First token on short prompts at 45 GB: 2.7-4.6 s; 2.26 s on chat as shipped, with PF-D [K]. Strata's is 0.5-1.0 s (0.83 s on chat at its full cache) [K].
- PF-D's speed was measured on chat-length prompts (37-101 tokens). On long prompts it also runs, once per layer and 1,024-token chunk, and stays exact: on 10K- and 30K-token prompts its tokens and logits hashes equal the same configuration without it, and a check arm found all 705,420 staged rows equal to the reference rebuild [K, BM:data/pfd_long/summary.txt]. There it shortens the first token by only 3-4 % (78.9 -> 76.6 s at 10K, 249.4 -> 238.4 s at 30K) and raises the lifetime peak by about 2 GB [K].
- Lifetime peaks above the memory line on long prompts [K]: 45.1-46.4 GB on a 7.9K-token prompt; on 10K / 30K-token prompts 45.2 / 46.7 GB without PF-D and 47.5 / 48.6 GB with it (the shipped 45 GB chat configuration) [K, BM:data/pfd_long/summary.txt], and 34.0 / 35.5 GB for the 32 GB chat configuration, 34.8 / 38.5 GB with chat.py's PF-A [K, LP:data/prefill_1109.md]. The 24 GB line stayed under 24 GB only on prompts of up to about 500 tokens in our runs, with little room (section 2): 25.0-28.2 GB at 3K-30K tokens [K, section 2], and 26.0-30.6 GB at 7K-30K tokens with chat.py's PF-A [K, LP:data/tue_afternoon_1110.md].
- At 32 GB the exact config was measured on chat (IFEval) only: its HumanEval, MBPP and math speeds were not measured. The 32 GB lossy config has IFEval rows of its own (L + COH) and code rows only from L before COH (C30).
- MLX version sensitivity (0.32.4 changes tokens against the exact references).
- Lossy configs are gated on the named sets only; the multilingual chat set fails the blunder line for M1-B (information).
- The clean-clone test ran on one MacBook Pro (section 3's) with two IFEval prompts per config (HumanEval for the 24 GB pair); other Macs and macOS versions are untested, and the 24 GB line was not run on a real 36 GB Mac (section 3).
- serve.py handles text only (no tool calls in 1.1.0) and runs one request at a time. Every request, and every chat.py turn, prefills its whole prompt: nothing is cached across requests or turns.

# 6. What we left on the table

- Chat prefill read-ahead: v4's next-layer prediction (layer L+1's router on layer L's input) supplied 71-75 % of the rows read from the SSD, and 19-31 % of what it staged went unused, so SSD bytes per prompt rose from 7.5 to 9.6 GB [K]. A better predictor, and PF-D measured on the other configs and at 32 GB, are left.

- SSD reads at ~7.3 GB/s [K] of a 12-14 GB/s drive; larger side pools; per-call Python overhead (~0.4 s per prompt) [K].

# 7. Credits and licences

- Strata (MIT; github.com/Niko1221/Strata; "Copyright (c) 2026 Niko1221 and the Strata contributors"): PF-D and its read-ahead follow Strata's prompt-path design (read only what memory lacks; read the next layer's misses during this layer). No Strata code is included; anything ported later keeps its notice.
- MTPLX by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx): our engine builds on its model code for this architecture, and it ran this model's own MTP head as a drafter on Apple Silicon before we did (its backend for the family is dated 29 Aug 2026). The package installs MTPLX from a pinned commit (it does not copy MTPLX's sources), ships its LICENSE and NOTICE (third_party/LICENSE-mtplx, NOTICE-mtplx), and, as the NOTICE requires, shows "Powered by MTPLX" and https://github.com/youssofal/mtplx in the CLI's startup banner and the README credits. Apache-2.0 is compatible with GPL-3.
- DFlash2 (Inco AI; Apache-2.0): every config drafts with the block drafter incoai/Qwen3.8-Flash-Next-DFlash2 (Hugging Face; a finetune of Qwen/Qwen3.8-Flash-Next; revision a22a174 of 19 Oct 2026, the copy our runs loaded), with our finetuned weights loaded over it. Inco AI, "DFlash 2: Keep Drafting Parallel" (blog, Aug 2026, inco.ai/blog/dflash2); the method: Chen, Liang and Liu, "DFlash: Block Diffusion for Flash Speculative Decoding" (ICML 2026; arXiv 2602.06036; github.com/z-lab/dflash, MIT). The package pins that revision (run.py passes it to the drafter's loader, section 4) and ships no weights; our finetuned weights (l120_ft.safetensors) are distributed separately, with Apache-2.0's licence text and a notice of our changes (third_party/LICENSE-dflash2, third_party/NOTICE-l120_ft).
- MLX (MIT), mlx-lm (MIT).
- The model: Qwen3.8-Flash-Next, "Copyright (c) 2026 Qwen", under the Qwen Community License 1.0. The weights and anything derived from them (the converted store too) stay under that licence, not the GPL-3. Its conditions bind whoever uses the weights: keep its notice; products or services with more than 100 million monthly active users or US$20 million monthly revenue must show the model's name; model-as-a-service and AI work assistant businesses need a separate licence from Qwen for commercial use. The 4-bit pack is MTPLX's (Hugging Face: Youssofal), under the same licence.

# 8. Sources (raw rows; the pooled values were re-derived from them; keys as in [MAIN] Appendix C)

Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps: BM: = BigMoE/, LP: = Leap/, LT: = a section of Leap/TABLE_350.md, MS: = MoonShots/, EV: = EvalSuite/docs/, SE: = StrataEval/. Commit hashes are those of each folder's own repository; the snapshot holds the files as last committed.

- 45 GB E / EB HumanEval, MBPP: BM:data/e5/ (E5 runs). E IFEval and math: BM:data/n45/ (N45, two passes; the shipped exact config, without SB1). L (= BM:data/stack/s5p_env.txt, CERT + the bundle): BM:data/s5p/ (S5' X passes: spx1-2, spmx1-2), BM:data/n45/ (IFEval, math: nil1-2, nml1-2; n45.sh reads s5p_env). CERT (row 28's label): S5' B passes (spb1-2, spmb1-2). M1-B + COH: CHS5 z0 / z0b (BM:data/chs/chs5_report.txt). 32 GB: BM:data/chs/c32_report.txt (e32, zs32). C30: BM:data/c30/ (f24_report.txt); its pooled rows from BM:data/r57/c30h1_160_runs.json + c30h2 (HumanEval) and c30m1 + c30m2 (MBPP12), pooled the way the re-derivation script ours_runs.py does (sum n / sum decode time; first token = pfs median). Disk sizes: the pack's safetensors headers (tensor bytes by name) and du of the converted store records3 (the converter's output; not in the public snapshot; 8 Nov 2026).
- Prefill: BM:data/pfd/pfd_report.txt at a6d5347 (part B; PF-A / PF-B), BM:data/pfb/pfb_report.txt (CT1), LP:data/pfa_tail_1111.md (PF-A on a short last chunk). PF-D: BM:data/pfd/ (v3 run, snapshot 7d50712; E rows), BM:data/pfd/pfd_report_2355.txt (diagnostic, PF-D first, chat control; snapshot 366c452), BM:data/pfd/pfd_report_0009.txt (v4 + read-ahead, FAST, E + v4 CHECK; snapshot 36ee120). Release freeze rel0: BM aa1ca3b; rel1: BM 0b4df8c (no change to the engine code BM:bigmoe/ since aa1ca3b).
- L + COH (45 and 32 GB): BM:data/rel/rel_rows_report.txt (REL ROWS on rel1, 9 Nov 09:47-09:57, quiet lock; data commit BM 53dfff5). Raw rows: BM:data/r57/rlh_256, rlm_256, rli_256, rlt_256 (C 256), rli_160 (C 160) and rlr_256 (the IFEval repeat), each _runs.json. Keys: HumanEval and MBPP as E5's e5h1 / e5m1 runs, IFEval and math = BM:data/n45/keys.txt lines 1 / 2. The IFEval cell at 45

GB pools rli_256 + rlr_256. The same names' _tokens.json are the package's lossy references. No row hit the screen's 600 s budget. The fan controller's log shows no heat pause in the run's window.

- Files read outside the freeze: the lossy and chat runs load three kernel files from the tree they were written in, not from the snapshot, and the runs' module manifests do not list them: MS:patches/planes_engine/tierglu.py (committed 2aeb5ad), gdn_exact.py and gdn_exact_patch.py (83109c8), all of 21 Oct 2026 with no later change (no later commit, clean tree). They are byte-identical (md5 dc81729b, 3b835480, 936c2635) to the copies frozen with the snapshot (BM:data/r57/snap/frozen_rel0/Leap_third_party/ms_gdn/) and to the package's kernels/ms_gdn/, which the package loads [K, LP:data/release/smoke_0700.txt]; the package's 9 Nov 07:00 smoke found the gap. The L + COH rows ran only after checking that the three files equal the copies frozen with rel1, and the same check passed after the run [K, BM:data/rel/rel_rows_report.txt].
- Chat as shipped vs M1-B + COH: tokens BM:data/r57/pdz4a_256_tokens.json (db922b9) == BM:data/r57/z0_256_tokens.json and zs_256_tokens.json (CHS5, 5abc51a) on 12 / 12 prompts; logits hashes ("lh") BM:data/r57/pdz4a_256_runs.json vs pdz0_256_runs.json 721 / 721; SB1 on E: BM:data/r57/yeh_256_runs.json vs yesh_256_runs.json 676 / 676 (CHS5). CHS5's z0 / zs runs carry no logits hashes, so the link from zs to M1-B + COH is token-level.
- Strata: SE:h2h (the head-to-head records); F:2026-11-08_1630_strata-head-to-head.
- 24 GB (1.1.0): the env line BM:data/stack/card_w24d_env.txt; the rows from BM:data/w24/ (w24d_report.txt for HumanEval, w24f_report.txt for MBPP and IFEval, w24g_report.txt for math) and their run records BM:data/r57/w13eh_100, w13fh_100, bf1m_100, bf2m_100, bf1i_100, bf2i_100, bg1a_100 and bg2a_100 (each _runs.json), pooled as above; the gate EV:RESULT_card_w24_2026-11-10; the package's build and checks LP:data/w24_pkg.md and LP:data/w24_pkg/; the simulated 36 GB Mac and the peaks by prompt length LP:data/tue_afternoon_1110.md and LP:data/tue_afternoon_1110/; Strata at 24 GB SE:h2h/results_low24.
- chat.py and serve.py (1.1.0): the code LP:release/src/chat.py and LP:release/src/serve.py (LP d37fb68); the checks of build 110g, whose code ships unchanged (the final package, build 110l, differs from it only in documents, its manifest and config comments: LP:data/w24_pkg/candidate_110k/diff_vs_110g.txt and LP:data/w24_pkg/candidate_110l/diff_vs_110k.txt), LP:data/w24_pkg.md and LP:data/w24_pkg/candidate_110g/checks_0345/ (the clean-clone run, the eight decode runs at 32 GB, the idle defaults and the heat-pause log); chat.py's changes and their tests LP:data/chatpy_fixes_1110.md and LP:data/chatpy_fixes_1110/. PF-A on the chat configuration: 32 GB LP:data/prefill_1109.md, runs LP:data/prefill_1109/ (cli_1110_000737 and 000918 at 10K, 001004 and 001438 at 30K, default and PF-A); 24 GB LP:data/tue_afternoon_1110.md sections 2 and 4, runs LP:data/tue_afternoon_1110/ (pfa24_default and pfa24_pfa.jsonl at 7K, p24_10k_* and p24_30k_*). The idle nap: LP:data/agent_cc_1111.md and LP:data/agent_cc_1111/idle_cpu_1111/ (idle CPU without it), LP:data/idle_ms_1111.md and LP:data/idle_ms_1111/ (with it, and the decode runs).

# Corrections

- **10 November 2026 (v1.0.1).** Section 5 said long prompts run without PF-D; they run with it, and it stays exact. Long prompts go over the memory lines (section 5). PF-B's exactness is limited to the tested prompts, up to 7.9K tokens.
- **11 November 2026 (v1.1.0).** Package 1.1.0 adds a 24 GB line (sections 1-5; [SMALL-RAM]), a chat in the terminal and a local server (sections 4 and 5). PF-B's difference on a 30K-token prompt comes from a short last chunk; a fix exists in a research copy, not in this package ([MAIN] section 6.2). PF-A, which runs only on prompts over 1,024 tokens, is exact only when their last 1,024-token chunk is about 200 tokens or longer (section 2). Section 1 now says the lifetime peak is measured against the memory line, which long prompts go over (section 5).

# Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

# References


- Youssof Altoukhi. MTPLX (software, v2.12.2, commit 9882703f; Apache-2.0). https://github.com/youssofal/mtplx
- Jian Chen, Yesheng Liang and Zhijian Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026. ICML 2026. Code: https://github.com/z-lab/dflash (MIT).
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos and Ronan Collobert. "MLX: Efficient and flexible machine learning on Apple silicon" (software, 2023; MIT). https://github.com/ml-explore/mlx
- Inco AI. "DFlash 2: Keep Drafting Parallel." Blog, Aug 2026. https://inco.ai/blog/dflash2
- Inco AI. Qwen3.8-Flash-Next-DFlash2 (drafter checkpoint, revision a22a174, 19 Oct 2026; Apache-2.0). https://huggingface.co/incoai/Qwen3.8-Flash-Next-DFlash2
- Quinn McNemar. "Note on the sampling error of the difference between correlated proportions or percentages." Psychometrika 12(2): 153-157, 1947. doi:10.1007/BF02295996.
- MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", 5 Oct 2026. https://github.com/ml-explore/mlx/issues/4632
- mlx-lm (software; MIT). https://github.com/ml-explore/mlx-lm
- Niko1221 and the Strata contributors. Strata (software, version 0.1.58-metal, commit 146df3c; MIT). https://github.com/Niko1221/Strata
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- Youssofal. Qwen3.8-Flash-Next-MTPLX-Bare-Speed (4-bit weights; Qwen Community License 1.0). https://huggingface.co/Youssofal/Qwen3.8-Flash-Next-MTPLX-Bare-Speed


Companion papers:


- [EVAL] Kamen Levi. Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac. 2026. A companion paper of this set.
- [MAIN] Kamen Levi. What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster. 2026. A companion paper of this set.
- [SMALL-RAM] Kamen Levi. A 125B mixture-of-experts model in 32, 24 and 12 GB of memory on one Mac. 2026. A companion paper of this set.